V FOODS — Complete Campus Dining & Pre-Order System Guide

> **A plain-English operational and architectural guide to the V FOODS platform.**
> Designed for students, canteen franchisees, university dining administrators, and technical partners.

Table of Contents

1. What is V FOODS?
2. Full-Mode System Architecture
3. The End-to-End Order Lifecycle
4. Wallet-First Payment Architecture
5. The Four Specialized Dashboards
6. Campus Canteens & Food Courts
7. Database Security & Row-Level Isolation (RLS)
8. Production Deployment & Infrastructure

1. What is V FOODS?

V FOODS is a smart food pre-ordering, prepaid digital wallet, and kitchen management platform custom-built for collegiate campuses and large festival dining grids.

The Problem on Campus

During 15-minute lecture breaks and lunch rush hours: - Hundreds of students crowd physical canteen counters simultaneously. - Payment terminals fail due to mobile network congestion or UPI timeouts. - Counter lines move slowly while cashiers manually handle payments, verify slips, and call out orders. - Students spend 25–40 minutes waiting in line, often missing meals or arriving late to classes.

The V FOODS Solution

1. **Pre-Order Ahead**: Students browse menus, customize items, and place pre-orders before walking to the food court.
2. **Prepaid Digital Wallet**: Payments are deducted in **under 20 milliseconds** from a pre-loaded campus wallet, eliminating checkout failure rates.
3. **Kitchen Display System (KDS)**: Kitchen staff receive orders digitally on a live board, prepare meals in advance, and package them hot.

4. **Instant Token Counter Collection**: Students receive a **4-digit digital token** (e.g. `#4826`) and QR pass. They arrive at the counter, show their token, pick up their hot meal in seconds, and leave. **Zero waiting in lines.**

---

2. Full-Mode System Architecture

The following diagram illustrates how all components of V FOODS communicate in real time across the university network:

```
flowchart TB
    subgraph Clients["CLIENT ACCESS LAYER (Progressive Web Application)"]
        direction TB
        C1["Student Interface<br/>(Menu Browse, Prepaid Wallet, Order Placement)"]
        C2["Kitchen Staff KDS<br/>(Live Order Tickets, Status Updates)"]
        C3["Canteen Owner Dashboard<br/>(Revenue Reports, Menu Management, Staffing)"]
        C4["Super Admin Console<br/>(Campus Oversight, Financial Reconciliation)"]
    end

    subgraph CDN["HOSTING & EDGE (Render Global CDN)"]
        direction TB
        Vite["Vite + React 19 Frontend<br/>(Cached globally, Offline Service Worker)"]
    end

    subgraph API["APPLICATION BACKEND (FastAPI / Python 3.11)"]
        direction TB
        F1["FastAPI Core Engine"]
        F2["Paytm Webhook Verifier"]
        F3["HMAC-SHA256 Token QR Engine"]
        F4["Background Task Scheduler"]
    end

    subgraph DataLayer["DATA & REALTIME LAYER (Supabase Cloud)"]
        direction TB
        Postgres[("PostgreSQL 15 Database")]
        RLS["Row-Level Security (RLS)"]
        Realtime["Realtime WebSockets (Live Sync)"]
        RPC["Atomic Stored Procedures<br/>(credit_wallet, place_order_wallet)"]
    end

    subgraph Gateways["EXTERNAL SERVICE GATEWAYS"]
        Paytm["Paytm Payment Gateway<br/>(UPI, NetBanking, Cards)"]
        MSG91["MSG91 Notification Gateway<br/>(WhatsApp & SMS Alerts)"]
    end

    %% Connections
    Clients -->|HTTPS Requests| Vite
    Vite -->|Client State| API
    Vite -->|Direct Read/Write| Postgres
    API -->|Service Role DB Access| Postgres
    Postgres --- RLS
    Postgres --- RPC
    Postgres -->|Live Database Changes| Realtime
    Realtime -.->|Instant UI Updates| Clients
    API -->|Initiate Payment| Paytm
    Paytm -->|Webhook Verification| API
    API -->|Order Ready / OTP| MSG91
    MSG91 -.->|SMS / WhatsApp| C1
```

3. The End-to-End Order Lifecycle

Every pre-order follows a structured 4-step pipeline designed for zero physical counter congestion:

```
sequenceDiagram
    autonumber
    actor Student as Student
    participant App as V FOODS Web App
    participant Wallet as Prepaid Wallet
    participant DB as Supabase PostgreSQL
    participant KDS as Kitchen KDS Display
    actor Staff as Kitchen Staff

    Student->>App: Browse Food Court & Select Dishes
    Student->>App: Select Timing (Immediate preparation or scheduled pickup slot)
    App->>Wallet: Verify Sufficient Balance or Gateway Authorization
    Wallet->>DB: Atomic RPC: Deduct Wallet & Create Order
    DB-->>App: Order Confirmed + 4-Digit Pickup Token (#4826)
    DB->>KDS: Realtime WebSocket: New Order Ticket Event (Audio Chime)
    Staff->>KDS: Tap 'Start Cooking' -> Status: PREPARING
    Staff->>KDS: Pack Food & Tap 'Mark Ready' -> Status: READY
    DB-->>App: Push & SMS Notification: 'Your Food is Ready at Counter'
    Student->>Staff: Show 4-Digit Token / QR Pass at Counter
    Staff->>KDS: Verify & Tap 'Mark Collected'
    DB-->>App: Order Completed & Archived
```

---

4. Dual Payment Architecture: Wallet + Instant Gateway

V FOODS provides flexible dual payment channels right at the checkout counter:

```
flowchart TD
    Cart["Student Cart Assembly"] --> Choice{"Select Payment Option"}

    %% Option 1: Campus Wallet
    Choice -->|Option 1: Prepaid Wallet| Wallet["V FOODS Campus Wallet"]
    Wallet -->|If Balance Sufficient| WPay["1-Tap Atomic Wallet Debit (< 20ms)"]
    Wallet -->|If Low Balance| WTop["1-Tap Top-Up Deficit via Paytm / UPI"]
    WTop --> WPay
    WPay --> OrderSuccess["Order Confirmed + Token (#4826) -> Kitchen KDS"]

    %% Option 2: Instant Gateway
    Choice -->|Option 2: Instant Gateway| Gateway["Instant Payment Gateway"]
    Gateway --> UPIOptions["Paytm UPI / GPay / PhonePe / Cards"]
    UPIOptions --> BankAuth["Direct Bank Authorization via Gateway"]
    BankAuth --> OrderSuccess
```

Payment Option Comparison:

Feature	Option A: V FOODS Campus Wallet	Option B: Instant Payment Gateway
Best For	Daily meals, regular students, 10-minute rush breaks	Visitors, first-time students, direct bank payers
Speed	**Under 20 milliseconds (Fastest)**	5–15 seconds (Standard UPI app authorization)
Prerequisites	Pre-loaded wallet balance (rechargeable anytime)	None; works with zero prior balance
Providers	Internal wallet float (recharged via Paytm / UPI)	Paytm PG, Google Pay, PhonePe, BHIM, Cards
Network Reliability	Works seamlessly even with spotty campus signals	Requires active mobile internet to authorize UPI
Deficit Support	Auto-calculated deficit top-up (+ ₹X & Pay)	Not needed — charges exact order total directly

---

5. The Four Specialized Dashboards

V FOODS provides 4 distinct interfaces tailored to specific responsibilities:

```
graph TD
    subgraph Roles["CAMPUS USER ACCESS LEVELS"]
        direction TB
        R1["1. Student / User<br/>• Mobile PWA Layout<br/>• 3-Column Food Grid<br/>• Left Rail Food
        R2["2. Kitchen Staff<br/>• Live KDS Kanban Board<br/>• Placed -> Preparing -> Ready<br/>• Co
        R3["3. Canteen Owner<br/>• Daily Gross & Net Revenue<br/>• Menu Catalog & Pricing Editor<br/
        R4["4. Super Admin<br/>• Campus-wide GMV & 5% Fee Ledger<br/>• 13 Canteen Health Matrix<br/>
    end
```

---

6. Campus Canteens & Food Courts

V FOODS models the physical layout of **VIT Chennai** into **5 Major Food Courts** housing **13 distinct counters**:

```
graph LR
    subgraph Campus["VIT CHENNAI CAMPUS"]
        FC1["Gazebo Food Court<br/>(Central Campus Plaza)"]
        FC2["North Square Food Court<br/>(North Square Building)"]
        FC3["AB3 Food Complex<br/>(Academic Block 3 Courtyard)"]
        FC4["Academic Blocks Diner<br/>(Delta & AB1 Walkways)"]
        FC5["Campus Pavilion & Stalls<br/>(Lakeview Activity Center)"]
    end

    FC1 --> G1["Gazebo C1 — Snacks & Fast Food"]
    FC1 --> G2["Gazebo C2 — Desserts & Sweets"]
    FC1 --> G3["Dakshin Chitra — South Meals"]
    FC1 --> G4["Lassi House — Shakes & Beverages"]

    FC2 --> N1["Georgia Canteen"]
    FC2 --> N2["Alpha Non-Veg Kitchen"]
    FC2 --> N3["Sri's Fast Food"]
    FC2 --> N4["Juice & Rice Corner"]

    FC3 --> AB3["AB3 Amphitheatre Kitchen"]
    FC4 --> AB1["AB1 Walkway Canteen"]
    FC4 --> DELTA["Delta Snacks Counter"]
    FC5 --> AV["Aavin Milk & Ice Cream Booth"]
    FC5 --> VM["V Mart Provisional Store"]
```

---

7. Database Security & Row-Level Isolation (RLS)

All permissions in V FOODS are secured directly by **PostgreSQL Row-Level Security (RLS)** in Supabase. The database rejects unauthorized queries at the engine level:

Role	Permitted Database Scope
Students	Read and update own profile, view personal wallet transactions and orders
Kitchen Staff	Read and update orders belonging strictly to their assigned outlet ID
Canteen Owners	Edit menu catalog, update prices, and view revenue strictly for their outlet ID
Super Admin	Full platform governance, campus-wide financial reconciliation, and system audits

Even if someone tampers with frontend code or runs unauthorized network commands, the database automatically filters out and blocks any record outside the user's validated session token.

---

8. Production Deployment & Infrastructure

V FOODS is deployed using automated continuous deployment on **Render.com**:

- **Frontend (`vfoods-web`)**:
- React 19 + Vite PWA static site deployed globally via Render's CDN.
- Automatically rebuilds on git pushes to `main` .
- **Backend API (`vfoods-api`)**:
- Python 3.11 FastAPI service deployed in the Singapore region (`uvicorn main:app`).
- Connects to Supabase PostgreSQL via connection pooling and verifies Paytm webhook signatures.
- **Database (`Supabase Cloud`)**:
- Hosted PostgreSQL 15 instance with automatic failover, WAL replication, and real-time subscription hubs.
